

# ENGR 102 Summer Bridge

## Day 11 Student Workbook

While Loops, Update Order, Sentinels, and Termination

Monday, July 20, 2026 • 50 points

|      |  |     |  |         |  |
|------|--|-----|--|---------|--|
| Name |  | UIN |  | Section |  |
|------|--|-----|--|---------|--|

**Boolean-operator convention.** Python operators appear as **and**, **or**, and **not**. Their lowercase spelling is required in code.

### Daily target

increasing independence

Design loops whose state changes make termination demonstrable.

|                         |                                                                            |
|-------------------------|----------------------------------------------------------------------------|
| <b>Allowed tools</b>    | Prior tools plus while loops and sentinel-controlled input                 |
| <b>Support supplied</b> | The sentinel, valid-data interval, and required summary are supplied.      |
| <b>You must decide</b>  | Loop pattern, update placement, validity branch, and the empty-data case.  |
| <b>Scaffold level</b>   | Termination evidence prompted; algorithm no longer sequenced line by line. |

### Scoring and completion standard

50 points

| Evidence               | Points | Secure work shows                                                   |
|------------------------|--------|---------------------------------------------------------------------|
| Integrated trace       | 12     | intermediate state, condition results, and exact output             |
| Diagnosis and repair   | 10     | actual, intended, cause, repair, and a boundary test when requested |
| Full program and tests | 28     | coherent executable logic, required output, and defended tests      |

**Independence standard.** You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

**Task 1. Integrated trace****12 points**

Trace both parts of the compound condition before each pass and the state after each update.

```
level = 17
steps = 0

while level < 50 and steps < 4:
    level = level + 9
    steps = steps + 1
    print(level, steps)

print("final", level, steps)
```

**Your task**

1. Build one ledger with condition result, level, and steps before and after each pass.
2. Give every output line and explain which condition evidence proves termination.

**Task 2. Diagnose and repair****10 points**

This loop should read integers until -1 and count nonnegative entries, but it can repeat forever.

```
reading = int(input())
valid_count = 0

while reading != -1:
    if reading >= 0:
        valid_count = valid_count + 1

print(valid_count)
```

**Your task**

1. Explain the infinite-loop condition using the unchanged state, not just the phrase missing update.
2. Write the minimal repair and describe the final false condition for inputs 5, 8, -1.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

**Task 3. Construct a complete program****28 points across Pages 4-5****Program specification**

- Read integer readings until the sentinel -1 is entered.
- Values from 0 through 100 inclusive are valid. Any other non-sentinel value is ignored and must print Ignored.
- For valid values, compute count, total, and the number at or above 70.
- After termination, print count, total, and high\_count on separate lines. Then print the average, or print No data when count is zero.

**Required planning evidence**

1. State the loop-controlling variable and show where it changes so that every path either terminates or reaches another input operation.

**Program workspace – begin the connected solution here.**

---

---

---

---

---

---

---

---

**Task 3 continued. Test and defend the program****included in 28 points**

Continue code if needed.

---

---

---

---

---

---

---

---

---

---

Required tests, expected results, and brief defense

|  |
|--|
|  |
|--|

# VIP Lab Alignment

## Bridge Day 11 • Contract 7cc3f8d502f3

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

### Why this page is here

**VIP Lab Day(s):** 5

**Activities:** 18.19, 18.21, 18.22

Complete the same while-loop cells with less planning support and defend the empty or already-finished case.

### Open these current notebook cells

| Activity / stable cells                                  | Notebook work                | Evidence to carry forward                         |
|----------------------------------------------------------|------------------------------|---------------------------------------------------|
| <b>18.19</b><br>18-19-work / 18-19-transfer / 18-19-cold | <b>Countdown To Multiple</b> | Write a while condition from a divisibility goal. |
| <b>18.21</b><br>18-21-work / 18-21-transfer / 18-21-cold | <b>Cooldown Sequence</b>     | Track a current value and a stored sequence.      |
| <b>18.22</b><br>18-22-work / 18-22-transfer / 18-22-cold | <b>Rounds To Clear Queue</b> | Model repeated queue reduction.                   |

### Readiness check before you code

1. Predict whether each loop executes zero, one, or multiple times for its transfer case before running it. \_\_\_\_\_  
\_\_\_\_\_
2. Identify where output or list append must occur relative to the update. \_\_\_\_\_  
\_\_\_\_\_

### Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook cold cells without a reveal or copied solution. Only those cold cells are scored for independent mastery on Lab Days 5–7.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.